SMART HOSPITAL APPOINTMENT BOOKING SYSTEM

A Mini Project Report Using Java

Submitted in partial fulfillment of the requirements for the course

CS25C09 - Java Programming

Submitted by

MALINI M

Register Number: 814725104086

Department of Computer Science and Engineering

SRM TRP Engineering College (An Autonomous Institution), Tiruchirappalli

Academic Year: 2026-2027

SRM TRP Engineering College
(An AUTONOMOUS Institution)
Tiruchirappalli
Higher Order Assessment with Rubrics

Department	CSE							
Programme:	B.E	**Semester:**	03	**AY:**	2026 – 2027 [ODD]			
Faculty Handling:	Mrs.R.Sasikala,AP/CSE							
Course Code & Title:	CS25C09 – Java Programming							
Assessment Task:	Project Based Learning	**Assessment Type:**	Individual Work	**Required Hrs:**	30			
Assessment Question based on:	*Open Ended*	✓	*Problem Based Learning*		*Creative Thinking*			
	Close Ended		*Project Based Learning*	✓	*Critical Thinking*	✓		
	Any Other Type		*Challenge Based Learning*		*Level of Thinking (RBT)*	K5/ K6		
SDG to be Addressed:	SDG is Required	Yes/ No	*SDG Number (Specify or Mention Students Choice)*		*Target Number (If Applicable)*			
SDG Integration Level: (If Addressed)	*Level 1: Awareness*		*Level 2: Design Alignment*	✓	*Level 3: Impact and Measurement*			
RIASEC Context	Realistic✓	Investigative ✓	Artistic	Social	Enterprising	Conventional		
Presentation Format:	Demonstration/Report/PowerPoint							
Max Marks:	**50**							
Relevant Course Outcome(s):	CO1, CO2, CO3, CO4, CO5							
Submission Deadline:	05.11.2026							
Submitted by (Name & Register No.):	Name: MALINI M Register No: 814725104086							
Project Title:	Smart Hospital Appointment Booking System							

Rubrics for Evaluation:

Assessment Criteria	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Framing (10)	9–10: Clearly defines the problem and objectives.	7–8: Defines the problem with minor gaps.	5–6: Shows basic understanding.	0–4: Problem is unclear.
Conceptual Foundation & Background Knowledge (10)	9–10: Demonstrates strong conceptual knowledge and relevant background.	7–8: Shows good understanding with minor gaps.	5–6: Demonstrates basic knowledge.	0–4: Shows insufficient knowledge.
Analytical Reasoning & Synthesis (15)	13–15: Analyses information logically and develops well-supported conclusions.	10–12: Provides reasonable analysis with minor gaps.	7–9: Provides basic analysis with limited justification.	0–6: Analysis is weak or incomplete.
Documentation & Clarity of Explanation (10)	9–10: Presents clear, structured ,complete documentation and upload the projects in Github Repository.	7–8: Documentation is clear with minor omissions.	5–6: Documentation is understandable but lacks detail.	0–4: Documentation is unclear or incomplete.
Reflective Learning Insight (5)	5: Clearly explains learning, challenges and improvements.	4: Explains learning with some reflection.	3: Identifies basic learning outcomes.	0–2: Provides little or no reflection.

Individual Mark Allocation:

Register No 814725104086		**Name:** MALINI M			
Assessment Criteria	**Excellent**	**Good**	**Satisfactory**	**Needs Improvement**	**Total**
Problem Understanding & Framing (10)					
Conceptual Foundation & Background Knowledge (10)					
Analytical Reasoning & Synthesis (15)					
Documentation & Clarity of Explanation (10)					
Reflective Learning Insight (5)					
				Grand Total	

Signature of the Faculty

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Existing and Proposed Systems

4. Hardware and Software Requirements

5. System Design

6. Java Concepts Used

7. Implementation

8. Sample Java Program

9. Sample Output

10. Testing

11. Analysis and Discussion

12. SDG Alignment

13. Advantages and Limitations

14. Conclusion

15. Future Enhancements

16. Reflective Learning Insight

17. GitHub Repository

18. References

19. Certificate

Abstract

Booking a doctor's appointment in many hospitals and clinics still means standing in a line at the reception, making a phone call that is not answered, or simply walking in and waiting for hours. Reception staff keep handwritten registers, two patients are often given the same time with the same doctor, and a patient who cancels leaves a slot that nobody else can use.

The Smart Hospital Appointment Booking System is a Java-based console application that digitises this process. The hospital's doctors are stored with their departments. A patient is registered at the time of booking, chooses a doctor, a date and a time slot, and receives a unique appointment ID. The system prevents double booking by never offering a slot that is already taken. Staff can view all appointments, search an appointment by its ID, cancel an appointment (which frees the slot again) and see simple statistics such as confirmed and cancelled appointments for each doctor.

The project applies object-oriented programming concepts such as classes, objects, constructors, encapsulation and methods, together with the Java Collections Framework (ArrayList and HashSet), the java.time date API, exception handling, loops, conditional statements and the Scanner class. The first version stores data in memory and uses a console interface, which keeps it suitable for beginners; it can later be extended with Java Swing, JDBC and SMS notifications. The project is aligned with UN Sustainable Development Goal 3 (Good Health and Well-being).

Chapter 1 – Introduction

A hospital outpatient department (OPD) serves a large number of patients every day. Each patient needs to meet a particular doctor at a convenient time and wants to be sure that the visit will actually take place at that time. When appointments are handled manually, the hospital faces overcrowded waiting halls, mismatched schedules and unhappy patients, while doctors face uneven workloads, with long idle periods followed by sudden rush hours.

In many small and medium hospitals, appointments are still noted in paper registers or given over the telephone. This depends fully on the receptionist, offers no instant view of free slots and makes it easy to book the same slot twice.

The Smart Hospital Appointment Booking System uses Java to automate the basic appointment operations. It lists the doctors, shows the free slots for a doctor on a chosen date, books and cancels appointments, avoids double booking and gives a quick summary of the day's work.

Problem Statement

Manual appointment handling in hospitals leads to double booking of the same time slot, lost or forgotten entries, no quick way to see which slots are free, wasted time for patients who arrive and find that their slot was given to someone else, and no record of how many appointments each doctor handles. Staff also spend time answering repeated questions such as "Is Dr. Kumar free tomorrow morning?"

Therefore, a simple Java application is proposed that stores doctor details, books appointments in free slots only, lets patients and staff search and cancel appointments, and reports basic statistics.

Objectives

- To develop a console-based hospital appointment booking application using Java.
- To maintain a list of doctors with their departments.
- To show the available time slots of a doctor on a given date.
- To book an appointment and generate a unique, sequential appointment ID.
- To prevent two patients from booking the same doctor, date and time slot.
- To search an appointment by its ID and to view all appointments.
- To cancel an appointment and release its slot for other patients.
- To validate the input (numbers and dates) so that wrong entries do not stop the program.
- To apply object-oriented programming concepts and the Collections Framework in a practical application.

Chapter 2 – Existing and Proposed Systems

Existing System

In the existing manual system:

- Appointments are written in a paper register or accepted over the telephone.
- Free and booked slots can be known only by reading through the register.
- Two patients can be given the same doctor and time by mistake.
- Cancelled slots are rarely reused, so the doctor's time is wasted.
- There is no record of how many patients each doctor consulted in a day.
- Patients must visit or call the hospital even to check or cancel an appointment.

Proposed System

The proposed Java application provides:

- A stored list of doctors and departments.
- Slot-wise availability for every doctor on every date.
- Automatic appointment ID generation for each booking.
- Automatic prevention of double booking.
- A menu-driven interface that is easy for reception staff to use.
- Appointment search, cancellation with slot release, and basic statistics.
- Scope for future enhancement using graphical interfaces, databases and SMS alerts.

Chapter 3 – Hardware and Software Requirements

Requirement	Specification
Hardware	Computer or laptop
Processor	Standard dual-core processor or better
RAM	4 GB or above recommended
Operating system	Windows or Linux
Programming language	Java
JDK	JDK 8 or later
IDE	Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Data storage	In-memory collections in the sample version
Version control	Git and GitHub repository

Chapter 4 – System Design

System Architecture

The main controller reads the user's choice from the menu and calls the corresponding module. The modules work on three data stores: the list of doctors, the list of appointments, and a set of booked slots that is used to detect double booking.

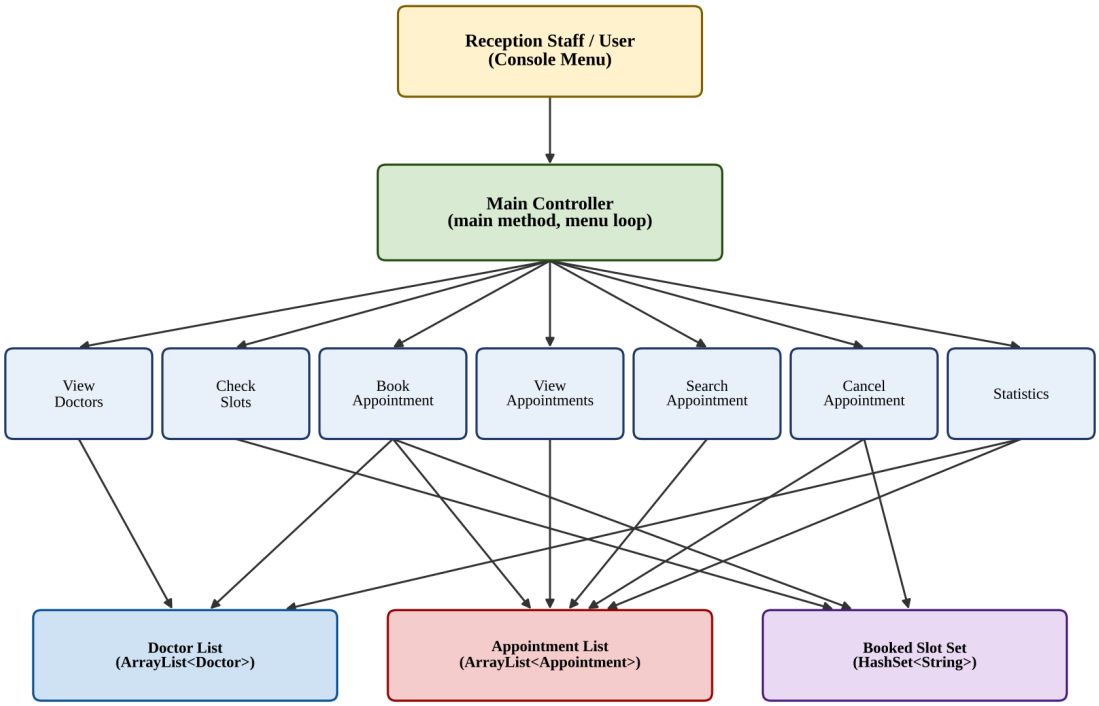


Fig. 4.1 System Architecture

Program Flow

The flow chart below shows how an appointment is booked, how the booked slot is protected against double booking, and how a cancellation releases the slot again.

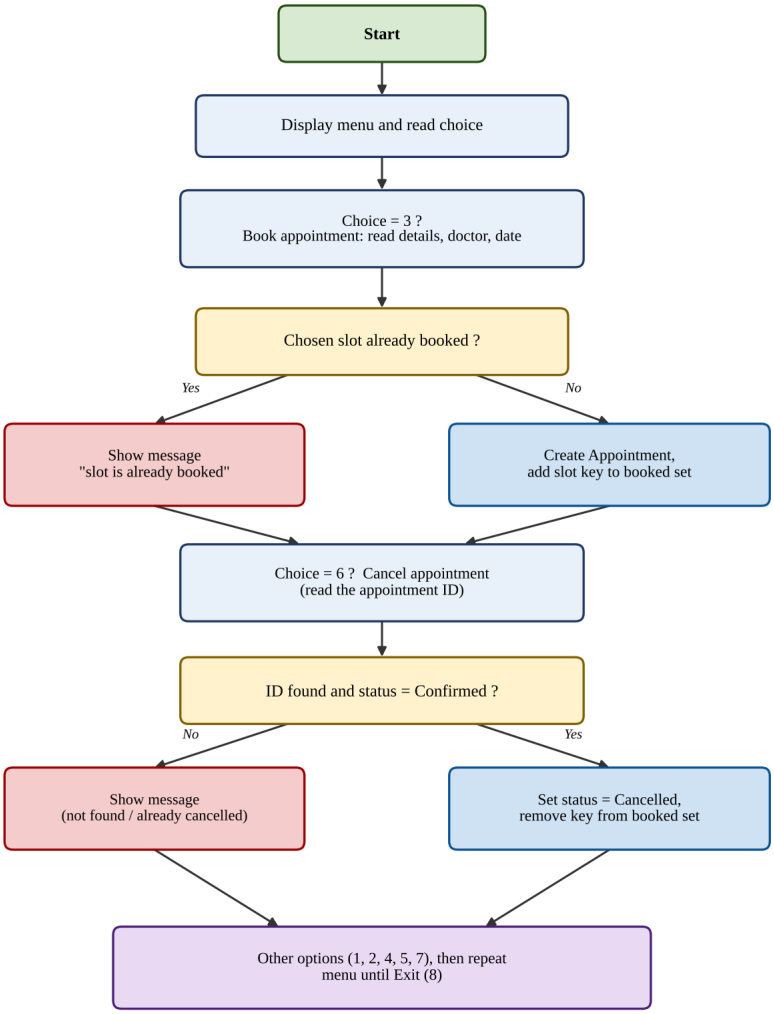


Fig. 4.2 Program Flow of the Appointment Booking System

Modules

Doctor Module

The Doctor class stores the doctor ID, name and department.

Appointment Module

The Appointment class stores the appointment ID, patient name, age, phone number, doctor, date, time slot and status (Confirmed or Cancelled).

Check Slots Module

Shows the free slots of a doctor on a date by checking each slot against the booked-slot set.

Booking Module

Accepts patient details, validates the input, checks that the slot is free, generates the next appointment ID and stores the appointment.

View and Search Module

Displays all appointments, or the appointment that matches a given ID.

Cancel Module

Changes the status to Cancelled and removes the slot from the booked-slot set.

Statistics Module

Shows the total, confirmed and cancelled appointments and the confirmed appointments of each doctor.

Chapter 5 – Java Concepts Used

Java concept	Application in the project
Class	Doctor, Appointment and SmartHospitalAppointment
Object	Individual doctor and appointment records created at booking
Encapsulation	Appointment information stored together as fields of one class
Constructor	Doctor(...) and Appointment(...) initialise their fields
Methods	Book, check slots, view, search, cancel and statistics operations
Association	Each Appointment holds a reference to its Doctor object
ArrayList	Stores the list of doctors and the list of appointments
HashSet	Stores booked slot keys; contains() detects double booking in constant time
String handling	Building the slot key "doctorId|date|slot" and comparing the status
java.time (LocalDate)	Parses and validates the date and rejects past dates
Exception handling	try-catch for NumberFormatException and DateTimeParseException
Enhanced for loop	Traverses the doctors and the appointments
Conditional statements	Decide the slot availability, the appointment status and the result of a search
switch and do-while	Menu selection and repeating the menu until Exit
Static members	Shared lists, booked-slot set, appointment counter and Scanner object
Scanner	Reads input from the user

Chapter 6 – Implementation

The application follows these steps:

1. Start the Java program and load the doctors.
2. Display the main menu.
3. Read the user's choice.
4. Execute the selected operation.
5. Display the result.
6. Return to the main menu until the user selects Exit.

Algorithm

Step 1: Start

Step 2: Create the doctor list, the appointment list and the booked-slot set, set the appointment counter to 1001 and load the doctors.

Step 3: Display the menu.

Step 4: Read the user's choice (ask again if it is not a number).

Step 5: If choice is 1, display all doctors.

Step 6: If choice is 2, read the doctor ID and the date and display the slots that are not in the booked-slot set.

Step 7: If choice is 3, read the patient details, doctor and date, display the free slots and read the slot number; if the slot key is already in the set, display "already booked", otherwise create the appointment, add it to the list, add the key to the set and display the appointment ID.

Step 8: If choice is 4, display all appointments.

Step 9: If choice is 5, read an appointment ID and display the matching appointment.

Step 10: If choice is 6, read an appointment ID, set its status to Cancelled and remove its key from the set.

Step 11: If choice is 7, display the total, confirmed and cancelled counts and the count of each doctor.

Step 12: If choice is 8, exit the program.

Step 13: Repeat the menu until Exit is selected.

Step 14: Stop.

Chapter 7 – Sample Java Program

The following is a complete console application. Save it in a file named SmartHospitalAppointment.java, compile it with javac SmartHospitalAppointment.java and run it with java SmartHospitalAppointment.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.Scanner;

class Doctor {
    int id;
    String name;
    String department;

    Doctor(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    void display() {
        System.out.println(id + " | Dr. " + name + " | " + department);
    }
}

class Appointment {
    int id;
    String patientName;
    int age;
    String phone;
    Doctor doctor;
    String date;
    String slot;
    String status;

    Appointment(int id, String patientName, int age, String phone,
                Doctor doctor, String date, String slot) {
        this.id = id;
        this.patientName = patientName;
        this.age = age;
        this.phone = phone;
        this.doctor = doctor;
        this.date = date;
        this.slot = slot;
        this.status = "Confirmed";
    }

    void display() {
        System.out.println(id + " | " + patientName + " | " + age + " | "
                + doctor.name + " (" + doctor.department + ") | "
                + date + " | " + slot + " | " + status);
    }
}

public class SmartHospitalAppointment {
    static final String[] SLOTS = {"09:00 AM", "10:00 AM", "11:00 AM",
                                   "02:00 PM", "03:00 PM", "04:00 PM"};
    static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("dd-MM-uuuu");

    static ArrayList<Doctor> doctors = new ArrayList<>();
    static ArrayList<Appointment> appointments = new ArrayList<>();
    static HashSet<String> bookedSlots = new HashSet<>();
    static Scanner sc = new Scanner(System.in);
    static int nextId = 1001;

    static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
```

```
            System.out.println("Please enter a valid number.");
        }
    }
}

static String readDate() {
    while (true) {
        System.out.print("Enter Date (dd-mm-yyyy): ");
        String text = sc.nextLine().trim();
        try {
            LocalDate d = LocalDate.parse(text, FMT);
            if (d.isBefore(LocalDate.now())) {
                System.out.println("Date cannot be in the past.");
            } else {
                return text;
            }
        } catch (DateTimeParseException e) {
            System.out.println("Invalid date format.");
        }
    }
}

static Doctor findDoctor(int id) {
    for (Doctor d : doctors) {
        if (d.id == id) {
            return d;
        }
    }
    return null;
}

static String key(int doctorId, String date, String slot) {
    return doctorId + "|" + date + "|" + slot;
}

static void loadDoctors() {
    doctors.add(new Doctor(1, "Ramesh Kumar", "Cardiology"));
    doctors.add(new Doctor(2, "Lakshmi Priya", "Orthopedics"));
    doctors.add(new Doctor(3, "Suresh Babu", "General Medicine"));
    doctors.add(new Doctor(4, "Anitha Raj", "Pediatrics"));
}

static void viewDoctors() {
    System.out.println("ID | Doctor | Department");
    for (Doctor d : doctors) {
        d.display();
    }
}

static void showAvailableSlots() {
    Doctor d = findDoctor(readInt("Enter Doctor ID: "));
    if (d == null) {
        System.out.println("Doctor not found.");
        return;
    }
    String date = readDate();
    System.out.println("Available slots for Dr. " + d.name + " on " + date + ":");
    boolean any = false;
    for (int i = 0; i < SLOTS.length; i++) {
        if (!bookedSlots.contains(key(d.id, date, SLOTS[i]))) {
            System.out.println((i + 1) + ". " + SLOTS[i]);
            any = true;
        }
    }
    if (!any) {
        System.out.println("No slots available on this date.");
    }
}

static void bookAppointment() {
    System.out.print("Enter Patient Name: ");
    String name = sc.nextLine().trim();
    int age = readInt("Enter Age: ");
    System.out.print("Enter Phone Number: ");
    String phone = sc.nextLine().trim();
    viewDoctors();
    Doctor d = findDoctor(readInt("Enter Doctor ID: "));
```

```
    if (d == null) {
        System.out.println("Doctor not found.");
        return;
    }
    String date = readDate();
    System.out.println("Available slots:");
    for (int i = 0; i < SLOTS.length; i++) {
        if (!bookedSlots.contains(key(d.id, date, SLOTS[i]))) {
            System.out.println((i + 1) + ". " + SLOTS[i]);
        }
    }
    int choice = readInt("Choose slot number: ");
    if (choice < 1 || choice > SLOTS.length) {
        System.out.println("Invalid slot number.");
        return;
    }
    String slot = SLOTS[choice - 1];
    String k = key(d.id, date, slot);
    if (bookedSlots.contains(k)) {
        System.out.println("Sorry, this slot is already booked.");
        return;
    }
    Appointment a = new Appointment(nextId++, name, age, phone, d, date, slot);
    appointments.add(a);
    bookedSlots.add(k);
    System.out.println("Appointment confirmed. Appointment ID: " + a.id);
}

static void viewAppointments() {
    if (appointments.isEmpty()) {
        System.out.println("No appointments found.");
        return;
    }
    System.out.println("ID | Patient | Age | Doctor | Date | Time | Status");
    for (Appointment a : appointments) {
        a.display();
    }
}

static void searchAppointment() {
    int id = readInt("Enter Appointment ID: ");
    for (Appointment a : appointments) {
        if (a.id == id) {
            a.display();
            return;
        }
    }
    System.out.println("Appointment not found.");
}

static void cancelAppointment() {
    int id = readInt("Enter Appointment ID to cancel: ");
    for (Appointment a : appointments) {
        if (a.id == id) {
            if (a.status.equals("Cancelled")) {
                System.out.println("Appointment is already cancelled.");
            } else {
                a.status = "Cancelled";
                bookedSlots.remove(key(a.doctor.id, a.date, a.slot));
                System.out.println("Appointment cancelled. Slot is free again.");
            }
            return;
        }
    }
    System.out.println("Appointment not found.");
}

static void showStatistics() {
    int confirmed = 0;
    int cancelled = 0;
    for (Appointment a : appointments) {
        if (a.status.equals("Confirmed")) {
            confirmed++;
        } else {
            cancelled++;
        }
    }
```

```
        System.out.println("Total appointments : " + appointments.size());
        System.out.println("Confirmed          : " + confirmed);
        System.out.println("Cancelled          : " + cancelled);
        for (Doctor d : doctors) {
            int count = 0;
            for (Appointment a : appointments) {
                if (a.doctor == d && a.status.equals("Confirmed")) {
                    count++;
                }
            }
            System.out.println("Dr. " + d.name + " : " + count + " confirmed");
        }
    }

    public static void main(String[] args) {
        loadDoctors();
        int choice;
        do {
            System.out.println("\n--- SMART HOSPITAL APPOINTMENT BOOKING ---");
            System.out.println("1. View Doctors");
            System.out.println("2. Check Available Slots");
            System.out.println("3. Book Appointment");
            System.out.println("4. View All Appointments");
            System.out.println("5. Search Appointment");
            System.out.println("6. Cancel Appointment");
            System.out.println("7. Statistics");
            System.out.println("8. Exit");
            choice = readInt("Enter your choice: ");
            switch (choice) {
                case 1: viewDoctors(); break;
                case 2: showAvailableSlots(); break;
                case 3: bookAppointment(); break;
                case 4: viewAppointments(); break;
                case 5: searchAppointment(); break;
                case 6: cancelAppointment(); break;
                case 7: showStatistics(); break;
                case 8: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice.");
            }
        } while (choice != 8);
        sc.close();
    }
}
```

Explanation of the Program

- Doctor and Appointment are classes whose constructors initialise the fields. An Appointment keeps a reference to the Doctor object, so the doctor's name and department are available when the appointment is displayed.
- doctors and appointments are ArrayList objects. bookedSlots is a HashSet of strings such as "1|15-12-2026|10:00 AM"; each string identifies one doctor, one date and one time slot.
- readInt() keeps asking until the user types a valid number, and readDate() uses LocalDate.parse() so that wrong formats and past dates are rejected. Every input is read with nextLine(), so the leftover newline problem of Scanner does not occur.
- bookAppointment() shows only the free slots and, before creating the Appointment, checks the set once more. This is what prevents double booking.
- cancelAppointment() changes the status to Cancelled and removes the key from the set, so the same slot can be booked again by another patient.
- showStatistics() counts the confirmed and cancelled appointments and the confirmed appointments of every doctor.
- The switch statement selects the operation and the do-while loop repeats the menu until the user exits.

Chapter 8 – Sample Output

Illustrative console output from the sample program (replace with your own screenshots after running the program):

```
--- SMART HOSPITAL APPOINTMENT BOOKING ---
1. View Doctors
2. Check Available Slots
3. Book Appointment
4. View All Appointments
5. Search Appointment
6. Cancel Appointment
7. Statistics
8. Exit
Enter your choice: 1
ID | Doctor | Department
1 | Dr. Ramesh Kumar | Cardiology
2 | Dr. Lakshmi Priya | Orthopedics
3 | Dr. Suresh Babu | General Medicine
4 | Dr. Anitha Raj | Pediatrics
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 3
Enter Patient Name: Ravi Kumar
Enter Age: 45
Enter Phone Number: 9876543210
(doctor list is displayed)
Enter Doctor ID: 1
Enter Date (dd-mm-yyyy): 15-12-2026
Available slots:
1. 09:00 AM
2. 10:00 AM
3. 11:00 AM
4. 02:00 PM
5. 03:00 PM
6. 04:00 PM
Choose slot number: 2
Appointment confirmed. Appointment ID: 1001
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 3
Enter Patient Name: Meena S
Enter Age: 30
Enter Phone Number: 9123456780
(doctor list is displayed)
Enter Doctor ID: 1
Enter Date (dd-mm-yyyy): 15-12-2026
Available slots:
1. 09:00 AM
3. 11:00 AM
4. 02:00 PM
5. 03:00 PM
6. 04:00 PM
Choose slot number: 2
Sorry, this slot is already booked.
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 3
(Meena S, 30, 9123456780, doctor 1 and 15-12-2026 entered again)
Choose slot number: 3
Appointment confirmed. Appointment ID: 1002
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 4
ID | Patient | Age | Doctor | Date | Time | Status
1001 | Ravi Kumar | 45 | Ramesh Kumar (Cardiology) | 15-12-2026 | 10:00 AM | Confirmed
1002 | Meena S | 30 | Ramesh Kumar (Cardiology) | 15-12-2026 | 11:00 AM | Confirmed
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 6
Enter Appointment ID to cancel: 1001
Appointment cancelled. Slot is free again.
--- SMART HOSPITAL APPOINTMENT BOOKING ---
Enter your choice: 7
Total appointments : 2
Confirmed          : 1
Cancelled          : 1
Dr. Ramesh Kumar : 1 confirmed
Dr. Lakshmi Priya : 0 confirmed
Dr. Suresh Babu : 0 confirmed
Dr. Anitha Raj : 0 confirmed
```

Observe that the second attempt to book the 10:00 AM slot with Dr. Ramesh Kumar on 15-12-2026 is rejected, and that after the cancellation of appointment 1001 the slot becomes available again.

Chapter 9 – Testing

Test case	Input / Action	Expected result
View doctors	Select option 1	All four doctors listed with departments
Book a free slot	Valid details, free slot	Appointment ID 1001 generated; status Confirmed
ID sequence	Book three appointments	IDs 1001, 1002 and 1003 issued in order
Double booking	Same doctor, date and slot again	"Sorry, this slot is already booked."
Same slot, different doctor	Same date and time, other doctor	Booking accepted
Check available slots	Doctor ID and date	Booked slots are not listed
Invalid doctor ID	Enter 9 as doctor ID	"Doctor not found."
Invalid slot number	Enter 0 or 7	"Invalid slot number."
Invalid date format	Enter 2026/12/15	"Invalid date format." and the date is asked again
Past date	Enter a date before today	"Date cannot be in the past."
Text instead of number	Enter "abc" for age or choice	"Please enter a valid number." and the question is repeated
Search existing ID	Enter 1001	Appointment details displayed
Search unknown ID	Enter 5555	"Appointment not found."
Cancel appointment	Enter a valid ID	Status Cancelled; slot free again
Cancel twice	Cancel the same ID again	"Appointment is already cancelled."
Statistics	Select option 7	Total, confirmed, cancelled and doctor-wise counts shown
Invalid menu choice	Enter 9	"Invalid choice."
Exit	Select option 8	Program terminates

These are expected test cases; the student should run the program and record the actual results.

Chapter 10 – Analysis and Discussion

Choice of Data Structures

The appointments are stored in an ArrayList because they are added at the end and displayed in the order of booking. The main design question was how to detect a double booking. A simple solution is to scan every appointment and compare the doctor, date and slot, but this takes linear time for every booking. A HashSet of keys of the form "doctorId|date|slot" answers the question "is this slot already taken?" in constant time on average, however many appointments are stored. A two-dimensional array indexed by doctor, day and slot was also considered, but it would need a fixed calendar and would waste memory for dates that are never used, whereas the set grows only with the real bookings.

Keeping the Set and the List Consistent

The appointment list keeps the history (including cancelled appointments, which are useful for statistics), while the set contains only the slots that are currently occupied. Therefore, whenever an appointment is booked its key is added to the set, and whenever it is cancelled the key is removed. Both updates happen in the same method, so the two structures cannot disagree.

Time Complexity

Operation	Complexity	Reason
Check one slot	O(1) average	HashSet contains()
Book appointment	O(1) average	Set lookup, then add to the end of the ArrayList
Show available slots	O(s)	One set lookup for each of the s slots (s = 6)
View all appointments	O(n)	Every appointment is displayed
Search appointment	O(n)	Linear scan of the list
Cancel appointment	O(n)	Linear scan to find the ID, O(1) to free the slot
Statistics	O(d × n)	For each of the d doctors the list is scanned

Observations and Limitations

All doctors have the same six fixed slots every day, and a slot is treated as one appointment. A real hospital has different timings for each doctor, holidays and leave. The system also keeps data in memory only, so the appointments are lost when the program ends, and the date check uses the computer's clock. For searching by ID a HashMap<Integer, Appointment> would give constant-time search and could replace the linear scan in a future version.

Chapter 11 – SDG Alignment

The project is aligned with UN Sustainable Development Goal 3 – Good Health and Well-being, in particular Target 3.8, which aims at access to quality essential health-care services. Giving patients a reliable, pre-booked time with a doctor reduces waiting and makes care easier to reach. The integration level is Level 2 (Design Alignment): the SDG influences the design decision of slot-based booking with strict prevention of double booking.

Design feature	**Contribution to SDG 3**
Slot-based booking with no double booking	Patients get a guaranteed time and do not wait for hours
Cancellation that releases the slot	Slots are reused, so more patients can be treated
Doctor-wise statistics	Helps management balance the workload of doctors
Simple console interface	Can be used by small clinics with basic computers

For Level 3 (Impact and Measurement), future versions can measure the average waiting time and the number of missed or wasted slots before and after using the system.

Chapter 12 – Advantages and Limitations

Advantages

- Easy to understand and use.
- Prevents double booking of a doctor's time slot.
- Removes dependence on paper registers and telephone calls.
- Cancelled slots are reused by other patients.
- Handles wrong input (text instead of numbers, wrong date format, past dates) without crashing.
- Demonstrates Java OOP concepts, the Collections Framework and exception handling.
- Can be extended with a graphical interface and database.

Limitations

- Appointment records are lost when the program terminates.
- Doctors and time slots are fixed in the program.
- There is no login system for reception staff or patients.
- Rescheduling is possible only by cancelling and booking again.
- The phone number is not validated and no reminder is sent.
- Only a console interface is available.

Chapter 13 – Conclusion

The Smart Hospital Appointment Booking System demonstrates how Java programming can be used to solve a real-world problem. The project implements doctor listing, slot availability, appointment booking with double-booking prevention, search, cancellation and statistics using classes, objects, constructors, methods, collections, exception handling, loops and conditional statements.

Through this project, the student gains practical experience in selecting suitable data structures, designing a menu-driven application, validating input, organising code into methods, and testing program functionality. The project also shows how a simple software solution can contribute to better healthcare service and serves as a foundation for developing larger Java applications.

Chapter 14 – Future Enhancements

The project can be improved by adding:

- Java Swing forms for a graphical user interface with a calendar and slot buttons.
- JDBC connectivity with MySQL for permanent storage of doctors, patients and appointments.
- Individual schedules for every doctor, including leave and holidays.
- A reschedule option and a waiting list for fully booked days.
- Patient and staff login with passwords.
- SMS or email confirmation and reminders before the appointment.
- Online booking through a website or mobile app.
- A HashMap for faster search by appointment ID.
- Reports on daily appointments, cancellations and peak hours.

Chapter 15 – Reflective Learning Insight

What I Learned

I learned how to model a real-world process with Java classes and collections, how a HashSet can detect conflicts much faster than scanning a list, and how a small design decision, such as storing a combined key for doctor, date and slot, can satisfy an important requirement. I also practised breaking a program into methods, validating input with exceptions, using the LocalDate class, using a menu loop and preparing test cases.

Challenges Faced

- Preventing two patients from booking the same slot, and keeping the booked-slot set correct when an appointment is cancelled.
- Handling wrong input, such as text instead of a number or a wrongly typed date, without the program crashing.
- Avoiding the leftover newline problem of Scanner by reading every input with nextLine().
- Deciding between scanning the list, a two-dimensional array and a HashSet for slot checking.

Improvements Planned

In the next version, I plan to add database storage, a Swing interface, separate timetables for each doctor, a reschedule option and SMS reminders, and to measure the actual reduction in waiting time.

GitHub Repository

As required by the assessment rubrics, the complete source code and this report are uploaded to a GitHub repository.

Repository link: https://github.com/<your-username>/SmartHospitalAppointment

Suggested repository contents:

- SmartHospitalAppointment.java – complete source code
- README.md – project description, steps to compile and run, and sample output
- Report (PDF/DOCX) and output screenshots

References

- Herbert Schildt, Java: The Complete Reference.
- Oracle Java Documentation: https://docs.oracle.com/en/java/
- Java Collections Framework: https://docs.oracle.com/javase/tutorial/collections/
- Java Date and Time API: https://docs.oracle.com/javase/tutorial/datetime/
- Java Exceptions: https://docs.oracle.com/javase/tutorial/essential/exceptions/
- Java Swing Tutorial: https://docs.oracle.com/javase/tutorial/uiswing/
- United Nations, Sustainable Development Goal 3: https://sdgs.un.org/goals/goal3

Course Completion Certificate



Course certificate – Introduction to Java (Sololearn), issued 09 August 2026